MATIS System

Error and Exception Flow Analysis

System Architecture Error Handling Assessment

Executive Summary

This analysis examines error handling patterns throughout the MATIS oncology analytics system, identifying both robust implementations and critical gaps that impact system reliability and debugging capabilities.

Error Flow Architecture

Controller Layer - HTTP Request/Response Handling
BaseController, API Controllers, Exception Filters

↓

Business Logic Layer - Domain Operations
Managers, Services, Validation Logic

↓

Data Access Layer - Persistence Operations
Repositories, Entity Framework, Dapper

↓

External Services - Third-party Integrations
HTTP Clients, Fax Services, Email, Salesforce

Controller Layer Error Handling

Strengths Identified

BaseController Validation

`BaseController.cs:39-56` - Implements structured input validation with consistent error messaging for user feedback operations.

```
public List<string> ValidateInputName(UserFeedbackModel feedback) { List<string> _errorMessage = new List<string>(); if (!CommonHelper.ValidateInput(feedback.Name, "String", null, 190, 1, true)) { _errorMessage.Add("Enter Name."); } return _errorMessage; }
```

Exception Handling with HTTP Status Codes

`AppealController.cs` - Proper exception catching with logging and appropriate HTTP response codes.

```
catch (ArgumentException ex) { _logger.Error(ex, ex.Message); return new HttpStatusCodeResult(HttpStatusCode.BadRequest, ex.Message); }
```

Critical Weaknesses

Minimal Base API Controller HIGH RISK

`BaseApiController.cs:4-14` - Lacks global exception handling middleware for consistent API error responses.

No Standardized Error Format MEDIUM RISK

Controllers return inconsistent error response structures across the application.

Business Logic Layer Error Propagation

Robust Implementation Examples

Assignment Audit Service

`AssignmentAuditService.cs:62,85` - Explicit validation with meaningful, domain-specific exceptions.

```
if (!AssignmentCanBeFound(assignmentWithChanges)) { throw new ArgumentException("Assignment cannot be found"); } if (fieldsThatLackTheChangeReason.Count > 0) { throw new SomeModifiedFieldsLackTheRequiredChangeReasonException(fieldsThatLackTheChangeReason); }
```

Problematic Patterns

Silent Failure Pattern HIGH RISK

`BreAgentClientProxy.cs:58-63` - HTTP client failures return null without error indication.

```
if (!response.IsSuccessStatusCode) { _logger.Error(response.ToString()); return null; // ⚠Silent failure - calling code cannot distinguish between // no data and service failure }
```

Boolean Return Anti-Pattern MEDIUM RISK

`ConfigurationManager.cs` - Methods returning true/false without error context or failure reasons.

Data Access Layer Issues

Critical Gaps Identified

- **Repository Pattern Inconsistency** - No standardized error handling for database operations across repositories
- **Entity Framework Context Management** - Missing comprehensive transaction rollback handling
- **Dapper Operations** - Limited error handling in raw SQL execution methods
- **Connection Management** - No retry logic for transient database failures

External Service Integration Vulnerabilities

HTTP Client Operations HIGH RISK

`AssignmentProviderManager.cs` - Missing timeout handling and connection failure recovery mechanisms.

Third-party Service Dependencies HIGH RISK

Fax services, email services, and Salesforce integration lack circuit breaker patterns and fallback mechanisms.

Risk Assessment: External service failures can cause cascading system failures due to lack of resilience patterns.

Logging Infrastructure Analysis

Well-Implemented Components

Structured Logging Framework

`NLogger.cs` - Comprehensive logging with exception details and structured data.

Correlation ID Support

`LoggerWithCorrelationId.cs` - Request tracing capabilities for distributed debugging.

Inconsistent Usage Patterns

Insufficient Context Logging - Many managers catch exceptions but don't log sufficient context for debugging.

Missing Correlation IDs - Error messages lack correlation IDs for tracking across services in many components.

File Operations and Document Management

Document Generation Pipeline HIGH RISK

Limited handling of file system errors, permission issues, and disk space constraints during document processing operations.

Impact: Potential data loss during critical document generation processes.

Recommendations for Improvement

Immediate Critical Fixes

- **Replace Null Return Anti-Pattern:** Implement Result<T> pattern or explicit exceptions in `BreAgentClientProxy.cs` and similar services
- **Global Exception Handling:** Add middleware for consistent API error responses across all controllers
- **Correlation ID Implementation:** Ensure all error logging includes correlation IDs for request tracing

Strategic Improvements

- **Circuit Breaker Pattern:** Implement for all external service calls (Salesforce, Fax services, Email)
- **Retry Policies:** Add configurable retry logic for transient failures in database and HTTP operations
- **Centralized Error Policies:** Create unified error handling policies across all application layers
- **Health Check Implementation:** Monitor dependency health and provide early warning systems

Monitoring and Observability

- **Error Rate Metrics:** Implement application-level error rate monitoring
- **Distributed Tracing:** Enhance correlation ID usage for full request lifecycle tracking
- **Alerting Mechanisms:** Set up proactive alerting for critical error patterns

Critical Risk Areas Summary

High-Priority Components Requiring Attention

- `ProtocolManager.cs:52` - Core business logic with complex error scenarios affecting patient care workflows
- `PatientManager.cs:40` - Patient data operations requiring maximum reliability and data integrity
- **External Service Integrations** - All third-party service communications lacking resilience patterns
- **Document Generation Pipeline** - File operations with potential for data loss during failures

System Reliability Impact

The identified error handling gaps present significant risks to system reliability, user experience, and debugging capabilities. Implementation of the recommended improvements will enhance system resilience and operational maintainability.